

Preffer Codebase Execution Flow Walkthrough

Interview Study Notes

Purpose

This PDF explains how the Preffer codebase actually runs internally. It is not a folder summary. It follows real execution paths: what triggers code, what data enters, what transforms, what database calls happen, what state changes happen, and what the user finally sees.

Mental Model Of The Whole App

Google login creates a local user.
The frontend stores a JWT token in localStorage.
apiFetch attaches that token to protected API calls.
Backend authenticate middleware verifies the token and attaches req.user.
wizard_data stores draft wizard state.
sections stores renderable public website content.
Publishing turns a draft into a public slug.
The public page renders sections by section.type.
The AI manager uses portfolio JSON as context.
Analytics records page views and chat messages.
AI capabilities turn conversations into structured business records.

1. Login Flow

User action:

The user clicks Start now on `frontend/src/app/page.tsx`.

Frontend function:

Home calls `loginWithGoogle()` from `frontend/src/context/AuthContext.tsx`.

Why this exists:

The browser cannot directly create an authenticated Preffer session. It first needs Google to prove the user's identity. After Google succeeds, Preffer creates its own local session using a JWT.

What `loginWithGoogle` does:

It checks if the user is inside the LinkedIn in-app browser. If yes, it redirects to `/open-in-browser`. Otherwise it sets:

```
window.location.href = API_URL + '/auth/google'
```

Request:

GET /api/auth/google

Backend route:

backend/src/routes/auth.routes.ts

```
router.get('/google', passport.authenticate('google', {
  scope: ['profile', 'email'],
  session: false
}))
```

Why Passport exists here:

Passport hides the OAuth redirect mechanics. It builds the Google login URL, sends the user to Google, and later receives the callback from Google.

Google callback:

After Google login succeeds, Google redirects to:

GET /api/auth/google/callback

Backend route:

backend/src/routes/auth.routes.ts

```
router.get('/google/callback',
  passport.authenticate('google', {
    session: false,
    failureRedirect: frontendBaseUrl + '/?error=auth_failed'
  }),
  AuthController.googleCallback
)
```

The key question:

But wait, where does the user record come from?

Passport strategy:

backend/src/config/passport.ts

Google returns a profile. The strategy extracts:

```
{
  googleId: profile.id,
  email: profile.emails[0].value,
  displayName: profile.displayName,
  profilePicture: profile.photos?.[0]?.value || null
}
```

Then it calls:

UserService.findOrCreateGoogleUser(googleUser)

Service:

backend/src/services/user.service.ts

If user exists:

```
SELECT * FROM users WHERE google_id = $1
```

Then:

```
UPDATE users
SET display_name = $1,
    profile_picture = $2,
    updated_at = NOW()
WHERE google_id = $3
RETURNING *
```

If user does not exist:

It creates a username from email, generates an API key, inserts a user, and logs signup credits.

```
INSERT INTO users (
  email,
  google_id,
  username,
  display_name,
  profile_picture,
  api_key
)
VALUES (...)
RETURNING *
```

Then:

```
INSERT INTO credit_transactions (
  user_id,
  amount,
  type,
  description
)
```

Why this matters:

Google only proves identity. It does not create Preffer credits, API keys, portfolio ownership, or local user rows. That is why the local users table exists.

AuthController.googleCallback:

backend/src/controllers/auth.controller.ts

It receives req.user from Passport and signs a JWT:

```
{
  userId: user.id,
```

```
email: user.email,  
username: user.username  
}
```

Then it redirects to:
`/auth/callback?token=<jwt>`

Frontend callback:
`frontend/src/app/auth/callback/CallbackInner.tsx`

It reads the token from URL, calls `setToken(token)`, and redirects to:
`/user/dashboard`

`AuthContext.setToken`:
`frontend/src/context/AuthContext.tsx`

```
localStorage.setItem('token', token)  
checkUser()
```

`checkUser` then calls:
`GET /api/auth/me`

That request goes through `apiFetch`.

`apiFetch`:
`frontend/src/lib/api.ts`

It reads:
`localStorage.getItem('token')`

Then sends:

Authorization: Bearer <token>
Content-Type: application/json

Backend authenticate middleware:
`backend/src/middleware/authenticate.ts`

It checks the Authorization header, verifies the JWT, and attaches:

```
req.user = {  
  userId: decoded.userId,  
  email: decoded.email,  
  username: decoded.username  
}
```

What would break without `authenticate`:
Protected routes would not know who owns the request. A user could fetch or mutate

another user's portfolio by guessing an ID.

Hidden complexity:

JWT logout is client-side only. `logout()` removes `localStorage` token and user. The backend does not revoke old tokens.

2. Dashboard Load Flow

User arrives at:

`/user/dashboard`

Component:

`frontend/src/app/user/dashboard/page.tsx`

`DashboardPage` waits for `AuthContext`. If auth is done and user exists, it calls `fetchPortfolios()`.

`fetchPortfolios` fires these in parallel:

`GET /api/portfolio/all`

`GET /api/portfolio/unfinished`

`GET /api/analytics/dashboard`

Why `Promise.all` is used:

These reads do not depend on each other. Running them in parallel makes dashboard load faster.

`GET /api/portfolio/all:`

Route: `backend/src/routes/portfolio.routes.ts`

Middleware: `authenticate`

Controller: `PortfolioController.getAllPortfolios`

Service: `PortfolioService.getByUserId`

SQL:

`SELECT * FROM portfolios`

`WHERE user_id = $1`

`ORDER BY updated_at DESC`

Data returned:

Array of portfolio rows, formatted by `PortfolioService.formatPortfolio`.

`GET /api/portfolio/unfinished:`

Controller: `PortfolioController.getUnfinishedPortfolios`

Service: `PortfolioChatService.getUnfinished`

SQL:

`SELECT id, name, portfolio_type, wizard_step, wizard_data, updated_at, created_at`

```
FROM portfolios
WHERE user_id = $1 AND status = 'draft'
ORDER BY updated_at DESC
```

GET /api/analytics/dashboard:

Controller: AnalyticsController.getDashboard

Service: AnalyticsService.getDashboardStats, getViewsPerDay, getMessagesPerDay, getTopPortfolios

Frontend state changes:

setPortfolios(allData)

setUnfinished(unfinishedData)

setAnalyticsStats(analyticsData.stats)

setLoading(false)

What rerenders:

The greeting, credits text, analytics cards, unfinished cards, and portfolio cards.

What would break without this:

The database could have portfolios, but the dashboard would look empty because React only renders from state.

3. Wizard Start Flow

User action:

User clicks Create New and lands on /user/wizard/new.

Main component:

frontend/src/app/user/wizard/[id]/page.tsx

Function: WizardPage

Important local state:

portfolio

loading

currentStep

tempWizardData

maxSections

Step 1 component:

StepType

User chooses:

'individual' or 'company'

StepType calls onNext(selected).

In WizardPage, because portfolioId is new, onNext calls createPortfolio(type).

Frontend request:

POST /api/wizard/start

Authorization: Bearer <jwt>

```
{  
  "portfolioType": "individual",  
  "name": "New Portfolio"  
}
```

Backend route:

backend/src/routes/wizard.routes.ts

router.use(authenticate)

router.post('/start', WizardController.startWizard)

Why router.use(authenticate) is important:

Every wizard endpoint requires a logged-in user. The wizard is not public because it creates and mutates user-owned portfolio data.

WizardController.startWizard:

backend/src/controllers/wizard.controller.ts

Inputs:

req.user.userId from authenticate

req.body.portfolioType

req.body.name

Validation:

portfolioType must be individual or company.

Credit and limit check:

CreditsService.getUserCredits(userId)

This checks:

user credits

plan

published portfolio count

draft portfolio count

Why drafts are limited:

Drafts are free, so without a draft limit a user could create endless unfinished rows.

Service call:

```
PortfolioService.createDraft(userId, {  
  portfolio_type: portfolioType,  
  name
```

```
}}
```

Database insert:

```
INSERT INTO portfolios (  
  user_id,  
  name,  
  portfolio_type,  
  wizard_step,  
  wizard_data  
)  
VALUES ($1, $2, $3, 2, $4)  
RETURNING *
```

wizard_data starts as:

```
{  
  "portfolioType": "individual"  
}
```

Why wizard_step is 2:

Step 1 is choosing the type. Once the row is created, step 1 is complete, so the saved next step is 2.

Response:

```
{  
  "portfolioId": "...",  
  "wizardStep": 2,  
  "portfolio": { ... }  
}
```

Frontend state changes:

```
setPortfolio(updatedPortfolio)  
setTempWizardData({ portfolioType })  
router.replace('/user/wizard/:id')
```

4. Wizard Data Model

Core database table:

portfolios

Important shape:

```
{  
  id,  
  user_id,  
  name,
```



```
slug,  
portfolio_type,  
profession,  
description,  
sections,  
theme,  
color_scheme,  
has_ai_manager,  
ai_manager_name,  
ai_manager_personality,  
ai_manager_has_portfolio_access,  
ai_manager_finalized,  
archestra_agent_id,  
status,  
wizard_step,  
wizard_data,  
chat_history,  
credits_used,  
created_at,  
updated_at  
}
```

Key mental model:

wizard_data is the notebook.

sections is the public website content.

Why this split exists:

During onboarding, the user may have partial choices: type selected, description written, sections recommended, theme chosen. Not all of that should render publicly. The public renderer mainly cares about sections, theme, slug, and published status.

5. About Step Flow

Component:

StepAboutAlways in frontend/src/app/user/wizard/[id]/page.tsx

Local state:

name

profession

description

documents

documentError

linkedinUrl

Validation:

name must exist

description must be at least 200 characters

User clicks continue.

StepAboutAlways calls:

onNext({ name, profession, description })

WizardPage parent calls:

updateWizardStep(3, data)

Request:

PATCH /api/wizard/:id/step/3

```
{
  "name": "...",
  "profession": "...",
  "description": "..."
}
```

Controller:

WizardController.updateWizardStep

Validation:

step must be 1 through 7

user must own portfolio

Service:

PortfolioService.updateWizard

What updateWizard does:

It first loads current portfolio by id and user_id.

Then:

```
const mergedData = { ...current.wizard_data, ...wizardData }
```

Then it writes:

UPDATE portfolios

SET wizard_step = \$1,

 wizard_data = \$2,

 updated_at = NOW()

WHERE id = \$3 AND user_id = \$4

RETURNING *

Why merge instead of replace:

If step 2 replaced wizard_data, it would erase step 1 portfolioType. Each wizard step adds fields to the same JSON object.

Response:

```
{
  portfolioId,
  wizardStep,
  wizardData,
  portfolio
}
```

Frontend:
setPortfolio(result.portfolio)
nextStep()

6. Supporting Documents Flow

Where used:
StepAboutAlways, StepSections, StepContent, and owner assistant chat.

Frontend helper:
uploadWizardDocument(portfolio.id, file)

Request:

POST /api/wizard/:id/documents

```
{
  "fileName": "...",
  "mimeType": "...",
  "dataUrl": "data:..."
}
```

Controller:
WizardController.uploadDocument

Validation:
fileName, mimeType, dataUrl required
dataUrl must start with data:
portfolio must exist and belong to user

Service:
InterfazeService.extractDocumentText

Then controller builds a stored document:

```
{
  id: "doc_<timestamp>_<random>",
  fileName,
  mimeType,
}
```

```
sizeBytes,  
extractedText,  
previewText,  
relevantSections,  
createdAt  
}
```

It merges into:

```
wizard_data.documentSources
```

Only last 5 documents are kept:

```
const mergedDocs = [...currentDocs, nextDoc].slice(-5)
```

Why this exists:

The AI generator needs context from resumes, docs, and LinkedIn data. The app stores extracted text, not the original file, so later AI calls can reuse the context.

Risk:

Extracted text is stored in JSONB inside portfolios. Large docs can make portfolio rows heavy.

7. Section Recommendation Flow

Component:

```
StepSections
```

Trigger:

useEffect runs getAIRecommendations() when there is a description and recommendations have not been fetched.

Request:

```
POST /api/wizard/:id/recommend
```

```
{  
  "description": wizardData.description  
}
```

Controller:

```
WizardController.recommendSections
```

Inputs:

```
portfolio type
```

```
profession
```

```
description
```

```
user plan
```

```
supporting document context
```

Service:

AIService.recommendSections

Prompt tells AI:

Must include hero first.

Must include contact last.

Recommend exactly max custom sections plus those two required sections.

Return JSON.

Expected AI response:

```
{  
  "sections": ["hero", "about", "services", "projects", "contact"],  
  "reasoning": "..."  
}
```

Backend cleanup:

It filters unknown section names.

It forces hero if missing.

It forces contact if missing.

It fills with defaults if too few sections.

Why backend does this:

AI output is not trusted blindly. The frontend renderer only supports known section types.

Frontend state changes:

setRecommendedSections(data.sections)

setSelectedSections(data.sections)

setMaxSections(data.maxSections)

setAiReasoning(data.reasoning)

setHasRecommended(true)

User can toggle sections:

Hero and contact cannot be removed.

Selected section count cannot exceed plan limit.

Continue request:

POST /api/wizard/:id/sections

```
{  
  "sections": ["hero", "about", "projects", "contact"]  
}
```

Controller:

WizardController.saveSections

Transforms strings into section objects:

```
{
  id: "section-<timestamp>-<index>",
  type,
  title,
  content: {},
  order: index
}
```

Service:

PortfolioService.updateSections

SQL:

```
UPDATE portfolios
SET sections = $1,
    updated_at = NOW()
WHERE id = $2 AND user_id = $3
RETURNING *
```

Then wizard_data gets:

```
{
  selectedSections: sectionTypes,
  currentSectionIndex: 0
}
```

8. Content Generation Flow

Component:

StepContent

Important local state:

currentIdx
chatMessages
conversationHistory
userInput
isGenerating
proposedContent
displayContent
isContentSaved
savedSections

When switching sections:

StepContent calls:

GET /api/wizard/:id/history/:sectionId

Backend:

WizardController.getChatHistory

PortfolioChatService.getChatHistory

SQL:

SELECT chat_history->\$1 as messages

FROM portfolios

WHERE id = \$2 AND user_id = \$3

Why chat_history is keyed by sectionId:

Each section has its own conversation. About section chat should not pollute Projects section chat.

Auto-generate trigger:

User clicks generate.

Frontend handleAutoGenerate sends:

POST /api/wizard/:id/generate

```
{  
  "sectionId": "section-..."  
}
```

Controller:

WizardController.generateContent

Steps:

1. Authenticate user.
2. Load portfolio.
3. Find section by sectionId.
4. Build context.
5. Call AIService.generateSectionContent.
6. Store generated content as pending.
7. Save chat history event.
8. Return proposed content to frontend.

Context shape:

```
{
  name,
  profession,
  description,
  portfolioType,
  documentContext,
  existingSections
}
```

AIService.generateSectionContent:

Builds a prompt containing:

profile data

existing saved sections

document context

section guidance

required section structure

Expected AI response:

```
{
  "content": { ...section-specific content... },
  "message": "..."
}
```

Important two-phase design:

The generated content is not saved to sections yet.

It is stored in:

pendingContentStore

Key:

portfolioId + '-' + sectionId

Value:

```
{
  sectionId,
  sectionType,
  content,
  proposedAt
}
```

Why this exists:

AI content should be reviewed by the user before becoming public website content.

Frontend response shape:


```
{
  "message": "...",
  "proposedContent": { ... },
  "displayContent": "...",
  "action": "proposal",
  "sectionType": "...",
  "requiresApproval": true
}
```

Frontend state changes:

```
setProposedContent(data.proposedContent)
setDisplayContent(...)
setIsContentSaved(false)
append AI message to chatMessages
```

User sees:

A draft preview and Approve & Save button.

9. Chat-Based Content Editing Flow

Trigger:

User types a message inside StepContent and sends.

Frontend function:

```
handleSendMessage
```

It optimistically appends:

```
{
  role: 'user',
  content: message,
  timestamp: new Date()
}
```

Then sends:

```
POST /api/wizard/:id/chat
```

```
{
  "sectionId": currentSection.id,
  "message": "...",
  "conversationHistory": [...]
}
```

Controller:

```
WizardController.chat
```

Flow:

1. Authenticate.
2. Validate sectionId and message.
3. Load portfolio.
4. Find target section.
5. Load saved chat history.
6. Build portfolio context.
7. Call AIService.chat.
8. Save updated chat history.
9. If AI proposed content, store pending content.
10. Return response.

AIService.chat expected response:

```
{  
  "message": "friendly natural text",  
  "action": "continue" | "proposal" | "ready",  
  "proposedContent": { ... },  
  "isComplete": true | false  
}
```

Action meanings:

continue means keep chatting.

proposal means show draft for approval.

ready means user approved something conversationally.

Frontend:

If proposedContent exists, it shows the draft and asks for approval.

If action is ready and proposedContent already exists, it calls handleApprove.

Risk:

conversationHistory from frontend can be stale because state updates are async. Backend reduces this by preferring saved DB history when available.

10. Approve Content Flow

Trigger:

User clicks Approve & Save.

Frontend function:
handleApprove

Request:

POST /api/wizard/:id/approve

```
{  
  "sectionId": "...",  
  "content": proposedContent  
}
```

Controller:
WizardController.approveContent

Flow:

1. Authenticate user.
2. Load portfolio.
3. Find section index.
4. Use request body content if present.
5. Otherwise fallback to pendingContentStore.
6. Parse JSON string if needed.
7. Sanitize project links or contact URLs.
8. Replace section content in sections array.
9. Save sections to DB.
10. Append approval message to chat_history.
11. Return saved content.

Section update shape:

```
updatedSections[sectionIndex] = {  
  ...section,  
  content: contentToSave  
}
```

SQL:

```
UPDATE portfolios
SET sections = $1,
    updated_at = NOW()
WHERE id = $2 AND user_id = $3
RETURNING *
```

Frontend state:

```
setSavedSections(prev => ({ ...prev, [currentSection.id]: true }))
setIsContentSaved(true)
refreshPortfolio()
```

Then after 1 second:

move to next section if one exists.

What would break without approval:

AI output could directly overwrite live content. Bad JSON or unsafe links could enter the portfolio without user review.

Scaling concern:

pendingContentStore is an in-memory Map. In a multi-server deployment, one request could generate pending content on server A, and approve could hit server B. This code avoids total failure because frontend sends content in the approve request, but relying on memory here is still fragile.

11. Features And AI Manager Setup

Component:

StepFeatures

Local state:

```
hasAiManager
aiManagerName
aiManagerPersonality
aiManagerHasPortfolioAccess
aiManagerFinalized
aiCapabilities
appointment settings
```

Why this exists:

The portfolio is not just a static page. The owner can add an AI representative that answers visitors and captures business intent.

On continue, frontend builds:

```
{
```

```
hasAiManager,  
aiManagerName,  
aiManagerPersonality,  
aiManagerHasPortfolioAccess,  
aiManagerFinalized,  
aiCapabilities: {  
  lead_capture: { enabled, settings },  
  appointment_requests: { enabled, settings },  
  order_quote_requests: { enabled, settings },  
  support_escalation: { enabled, settings },  
  faq_unknown_escalation: { enabled, settings },  
  follow_up_requests: { enabled, settings },  
  feedback_reviews: { enabled, settings }  
}  
}
```

Request:

PATCH /api/wizard/:id/step/6

Controller:

WizardController.updateWizardStep

If aiCapabilities is present:

It normalizes each enabled capability and calls:

AICapabilityService.upsertCapabilities

DB table:

portfolio_ai_capabilities

Columns:

portfolio_id

capability_key

enabled

settings_json

Why it is stored twice:

The app stores capability config in portfolio_ai_capabilities for querying and also mirrors it into wizard_data for frontend compatibility.

12. Theme Step

Component:

StepTheme

User chooses:

theme

colorScheme

Request:

PATCH /api/wizard/:id/step/7

Payload:

```
{
  "theme": "...",
  "colorScheme": {
    "name": "...",
    "colors": ["#...", "#...", "#...", "#..."]
  }
}
```

Backend:

PortfolioService.updateWizard merges this into wizard_data.

Important:

Before publish, theme mostly lives in wizard_data. On publish, PortfolioService.publish writes theme into the portfolio root column too.

13. Publish Flow

Component:

StepPublish

Slug check:

POST /api/wizard/:id/slug-check

```
{
  "slug": "my-name"
}
```

Controller:

WizardController.checkSlug

Service:

PortfolioService.isSlugAvailable

SQL:

SELECT id FROM portfolios WHERE slug = \$1 AND id != \$2

Publish request:

POST /api/wizard/:id/publish

```
{
  "slug": "my-name",
  "hasAiManager": true
}
```

Controller:

WizardController.publishPortfolio

Validation:

slug is required

slug must match lowercase letters, numbers, hyphens

slug must be available

user must have enough credits

Cost:

CreditsService.getPortfolioCost(hasAiManager)

Basic portfolio: 100 credits

With AI manager: 250 credits

Service:

PortfolioService.publish

Important transaction:

It locks the portfolio row:

```
SELECT * FROM portfolios
WHERE id = $1 AND user_id = $2
FOR UPDATE
```

Then updates:

status = 'published'

slug = slug

has_ai_manager = effectiveHasAiManager

name = wizardData.name

profession = wizardData.profession

description = wizardData.description

credits_used = cost

ai_manager_name = wizardData.aiManagerName

ai_manager_personality = wizardData.aiManagerPersonality

ai_manager_has_portfolio_access = wizardData.aiManagerHasPortfolioAccess

ai_manager_finalized = wizardData.aiManagerFinalized

theme = wizardData.theme || 'minimal'

wizard_step = 0

Then credits are deducted:

CreditsService.deductCredits

Credit deduction:

Locks user row with:

```
SELECT credits FROM users WHERE id = $1 FOR UPDATE
```

Then:

```
UPDATE users SET credits = credits - $1
```

And logs:

```
INSERT INTO credit_transactions
```

Hidden issue:

Portfolio publish commits first, then credits are deducted in a separate service transaction. If credit deduction fails, it manually rolls back portfolio status. A single DB transaction would be cleaner.

Archestra background operation:

If AI manager is finalized and Orchestra is enabled:

```
ArchestraAgentService.createAgentOrFallback
```

Then the agent id is saved:

```
archestra_agent_id
```

Response:

```
{
  portfolioId,
  slug,
  status,
  creditsUsed,
  url,
  message
}
```

Frontend redirects to dashboard.

14. Public Portfolio Render Flow

Visitor opens:

```
/:slug
```

Component:

```
frontend/src/app/[slug]/page.tsx
```

```
PublicPortfolioPage
```

Trigger:

useEffect calls fetchPortfolio when slug exists.

Request:

GET /api/portfolio/slug/:slug

Public route:

No auth required.

Controller:

PortfolioController.getBySlug

Service:

PortfolioService.getBySlug

SQL:

```
SELECT * FROM portfolios
WHERE slug = $1 AND status = 'published'
```

Why status check matters:

Draft portfolios should not be publicly visible even if someone guesses the slug.

Frontend state:

setPortfolio(data)

setLoading(false)

Theme creation:

PublicPortfolioPage uses:

getThemeForPortfolio(portfolio.theme, colorScheme)

If dark mode is enabled:

invertColorScheme is used and theme is recomputed.

Visible section filtering:

```
portfolio.sections
  .filter(s => s.visible !== false && sectionHasContent(s))
  .sort((a, b) => a.order - b.order)
```

Then contact section is moved last.

Why sectionHasContent exists:

The wizard creates empty sections first. The public page should not render empty placeholders.

Rendering:

renderSection(section, theme, aiManagerName, aiManagerPath, socialLinks)

Component map:

hero -> HeroSection
about -> AboutSection
services -> ServicesSection
skills -> SkillsSection
experience -> ExperienceSection
projects -> ProjectsSection
testimonials -> TestimonialsSection
contact -> ContactSection
faq -> FaqSection
pricing -> PricingSection
team -> TeamSection
menu -> MenuSection
achievements -> AchievementsSection
education -> EducationSection

What would break:

If AI generates a section type not in the map, renderSection returns null and the section disappears.

Analytics side effect:

When portfolio loads, frontend fires:

POST /api/analytics/page-view

```
{  
  "slug": "..."  
}
```

This is fire-and-forget.

Backend:

AnalyticsController.recordPageView

PortfolioService.getBySlug

AnalyticsService.recordPageView

SQL:

```
INSERT INTO portfolio_page_views (  
  portfolio_id,  
  visitor_ip,  
  user_agent,  
  referrer  
)
```

Why fire-and-forget:

Analytics should not block the visitor from seeing the page.

15. Public AI Manager Chat Flow

Visitor clicks:

Chat with <ai_manager_name>

Route:

/:slug/aiManagerName

Component:

frontend/src/app/[slug]/[aiManagerName]/page.tsx

AiManagerPage

Metadata fetch:

GET /api/portfolio/slug/:slug/ai-manager/:aiManagerName

Controller:

PortfolioController.getPublicAiManager

Validation:

Portfolio exists.

has_ai_manager is true.

ai_manager_finalized is true.

ai_manager_name exists.

Route name matches stored manager name after normalization.

Response:

```
{
  portfolio: {
    id,
    slug,
    name,
    theme,
    color_scheme,
    wizard_data
  },
  aiManager: {
    name,
    personality
  },
  greeting
}
```

Sending a message:

Frontend handleSend appends user message to local state and sends:

POST /api/portfolio/slug/:slug/ai-manager/:aiManagerName/chat

Headers:

Content-Type: application/json

x-preffer-session: <sessionId if known>

Body:

```
{  
  "message": "Can I hire you?",  
  "sessionId": "optional"  
}
```

Controller:

PortfolioController.chatWithPublicAiManager

Flow:

1. Validate message.
2. Load portfolio by slug.
3. Validate AI manager route.
4. Resolve sessionId from body, header, or crypto.randomUUID().
5. Set response header x-preffer-session.
6. Load session history.
7. Build analytics context.
8. Use Archestra A2A if linked and enabled.
9. Otherwise use directGeminiChat.
10. Record chat messages.
11. Trigger capability capture in background.
12. Return reply.

Session history:

AnalyticsService.getSessionHistory(portfolio.id, sessionId, 12)

SQL:

Find session by portfolio_id and session_key.
Then get last messages from ai_manager_messages.

Why sessionId exists:

IP-based sessions are weak. sessionId lets the browser continue the same conversation even if IP changes or multiple visitors share an IP.

directGeminiChat:

Builds portfolio context from:

- portfolio name
- profession
- description
- all saved sections with content
- custom AI instructions
- enabled capabilities
- analytics context
- recent conversation

Prompt asks Gemini to return:

```
{  
  "reply": "..."  
}
```

Response to frontend:

```
{  
  "reply": "...",  
  "aiManager": {  
    "name": "...",  
    "personality": "..."  
  },  
  "sessionId": "..."  
}
```

Frontend state:

```
setMessages([...updatedMessages, { role: 'assistant', content: data.reply }])  
setSessionId(nextSessionId)  
sessionStorage.setItem('ai_session:slug:name', nextSessionId)
```

What user sees:

The AI reply appears in the chat UI.

16. AI Capability Capture Flow

This is the hidden business pipeline.

After AI manager replies, backend does:

```
AnalyticsService.recordChatMessage(portfolio.id, visitorIp, 'visitor', message,  
sessionId)
```

```
AnalyticsService.recordChatMessage(portfolio.id, visitorIp, 'ai', finalReply, sessionId)
```

Then:

```
AICapabilityService.captureFromConversation(...)
```

This runs in background.

Why background:

The visitor should not wait for extraction, database inserts, and optional email sending before getting the AI reply.

captureFromConversation flow:

1. Load enabled capabilities for portfolio.
2. If none enabled, return.
3. Trim visitor message and AI reply.
4. Ask AI to extract actions.
5. Add heuristic lead capture fallback for strong hiring intent.
6. For each action:
 - ensure capability is enabled
 - map extracted fields to table columns
 - check readiness rules
 - create or update record
 - log tool event
 - optionally send lead follow-up email

Capability tables:

lead_capture -> ai_leads

appointment_requests -> ai_appointments

order_quote_requests -> ai_orders

support_escalation -> ai_support_tickets

faq_unknown_escalation -> ai_faq_unknowns

follow_up_requests -> ai_followups

feedback_reviews -> ai_feedback

Tool audit table:

ai_tool_events

Why ai_tool_events matters:

It creates an audit trail of what automated action happened, what payload triggered it, and whether it succeeded.

Idempotency:

Records use `idempotency_key` to avoid duplicate inserts for repeated or retried actions.

Readiness rules:

Some capabilities require contact info. For example, appointment requests require contact and either `requested_datetime` or `reason`. This prevents low-quality half-records.

17. Owner Assistant Chat Flow

Route:

`/user/chat`

Frontend:

`frontend/src/app/user/chat/page.tsx`

Initial load:

`loadInitial()` calls:

`GET /api/assistant/context-options`

`GET /api/assistant/chats`

Backend:

`assistant.routes.ts` uses `router.use(authenticate)`, so all owner assistant routes are protected.

Starting a chat:

Frontend sends:

`POST /api/assistant/chats`

```
{
  "contextType": "portfolio",
  "portfolioId": "...",
}
```

Controller:

`AssistantController.createChat`

Service:

`AssistantChatService.createChat`

DB insert:

`INSERT INTO assistant_chats (`

```
user_id,  
context_type,  
portfolio_id,  
title  
)  
RETURNING *
```

Then it inserts an intro assistant message into assistant_chat_messages.

Sending a message:

POST /api/assistant/chats/:chatId/messages

```
{  
  "message": "Make the projects section more professional"  
}
```

Controller:

AssistantController.sendMessage

Flow:

1. Authenticate user.
2. Validate message.
3. Load chat by id and user_id.
4. Insert user message.
5. AssistantChatService.handleUserMessage.
6. Maybe auto-title the chat.
7. Return userMessage and assistantMessage.

Portfolio content edits:

AssistantChatService does not save content edits immediately. It creates a pending proposal message with metadata:

```
{  
  status: "pending_portfolio_proposal",  
  sectionId,  
  sectionType,  
  proposedContent  
}
```

Frontend detects that metadata and shows Approve Changes.

Approving:

POST /api/assistant/chats/:chatId/approve

```
{  
  "proposalMessageId": "..."  
}
```

Backend finds unresolved pending proposal and applies it by updating portfolio.sections.

Why this mirrors wizard approval:

AI changes to public content still need human approval.

18. Analytics Dashboard Flow

Dashboard page calls:

GET /api/analytics/dashboard

Protected route:

authenticate -> AnalyticsController.getDashboard

Service calls in parallel:

getDashboardStats

getViewsPerDay

getMessagesPerDay

getTopPortfolios

getDashboardStats uses lateral joins:

portfolio_page_views

ai_manager_sessions

ai_manager_messages

Returned shape:

```
{  
  stats: {  
    total_views,  
    unique_visitors,  
    total_sessions,  
    total_messages,  
    views_5d,  
    sessions_5d,  
    messages_5d,  
    avg_messages_per_session,  
    visitor_to_chat_rate  
  },  
}
```

```
viewsPerDay,  
messagesPerDay,  
topPortfolios  
}
```

Why analytics are joined through portfolios:
The owner should only see analytics for portfolios they own.

AI insights:
GET /api/analytics/insights

AnalyticsService.generateInsights:

1. Gets stats.
2. Gets top portfolios.
3. Gets recent conversations.
4. Summarizes conversations in batches.
5. Runs a second AI call to produce business intelligence.

This is a two-phase AI pipeline:
First summarize raw conversations.
Then generate insights from summaries and stats.

Why two phases:
Raw conversations can be too long for one prompt. Batching reduces context size and makes the final insight prompt cleaner.

19. Important Design Choices

Strong choices:

JWT auth is simple and stateless.
wizard_data and sections separate draft state from public content.
AI-generated content requires approval.
Public page rendering is data-driven through section.type.
Analytics writes are fire-and-forget.
Public AI chat falls back to Gemini if Archestra fails.
AI capability records use idempotency keys.

Weak spots and interview talking points:

Duplicate route:

backend/src/routes/auth.routes.ts defines /credits twice.

Section type mismatch:

backend/src/types/section.types.ts has older section types like text_block and card_grid.

frontend/src/types/section.types.ts uses the 14 wizard section types.

This can confuse future maintainers.

In-memory pending store:

pendingContentStore is a Map in the server process.

This is risky for multi-instance deployments and restarts.

Better: store pending proposals in DB or Redis.

Publish transaction:

Portfolio publish and credit deduction happen in separate transactions.

Better: do both under one DB client transaction.

Inconsistent frontend API style:

Some files use apiFetch.

Wizard uses raw fetch often.

This leads to inconsistent error handling and auth behavior.

Heavy JSONB rows:

sections, wizard_data, documentSources, and chat_history all live on portfolios.

This is simple for a hackathon app but may become heavy as documents and chat histories grow.

20. Quick Interview Script

If asked: How does auth work?

The Start now button redirects to backend Google OAuth. Passport validates Google and finds or creates a local user. The backend signs a JWT containing userId, email, and username, redirects back to the frontend callback, and the frontend stores it in localStorage. Later apiFetch attaches it as Bearer token. Backend authenticate verifies it and puts userId on req.user for protected routes.

If asked: How does portfolio creation work?

The wizard starts with /api/wizard/start. The backend checks user limits, creates a draft portfolio row with wizard_step 2 and wizard_data containing portfolioType. Later steps merge more fields into wizard_data. Selected sections become objects in sections JSONB with empty content. AI fills them only after user approval.

If asked: How does AI-generated content get saved?

Generate or chat calls AIService, which returns proposedContent. The controller stores it as pending and returns it to the frontend. The user reviews it and clicks approve.

Only then approveContent replaces the matching section's content in the sections array and saves it to PostgreSQL.

If asked: How does the public page render?

The public route fetches `/api/portfolio/slug/:slug`. Backend returns only published portfolios. Frontend filters non-empty sections, sorts them, moves contact last, then calls `renderSection`. `renderSection` maps `section.type` to the correct React component.

If asked: How does AI manager chat work?

The public AI chat page validates the slug and AI manager name, sends visitor messages to the backend, loads recent session history, builds portfolio context from saved sections, routes through Archestra if possible or Gemini fallback, returns a reply, records analytics, and triggers background capability extraction.

If asked: What is clever here?

The app treats portfolio content as structured JSON sections. That allows one renderer to support many website types. It also uses a two-phase AI approval model so AI drafts do not immediately mutate public content.

If asked: What would you improve?

Move pending proposals from memory to DB or Redis, unify section type definitions, make publish and credit deduction one transaction, standardize frontend API calls on `apiFetch`, and split heavy chat/document data out of the portfolios row as the app scales.